

# AERO++

## Installation and Build Guide on Windows with MSYS2 MinGW64

---

English edition — revised and updated

This guide walks you through installing the AERO++ toolchain on Windows using MSYS2 MinGW64. It is targeted at students starting from a clean MSYS2 installation. The recommended project location is your MSYS2 home directory (`~/`), which on Windows corresponds to `C:\msys64\home\<user>\`. **Do not place the project under `C:\Users\<user>\Documents`**, because that path is typically synchronised with OneDrive and can silently remove or corrupt build artefacts produced by CMake and Ninja.

## Table of contents

1. Step 1 — Open the correct terminal
2. Step 2 — Update MSYS2
3. Step 3 — Install toolchain and base libraries
4. Step 4 — Install VTK and useful dependencies
5. Step 5 — Clone AERO++ into the MSYS2 home
6. Step 6 — Vendor OpenXLSX at the correct commit
7. Step 7 — Sanity-check the CMakeLists.txt
8. Step 8 — Configure CMake
9. Step 9 — Build (full or single target)
10. Step 10 — Run an executable
11. Quick reference — full copy-paste sequence
12. Troubleshooting
13. Final rules

## Step 1 — Open the correct terminal

Open **MSYS2 MinGW 64-bit** from the Start menu. The prompt must display MINGW64:

```
amedeo@Amedeo MINGW64 ~
```

Other MSYS2 environments such as UCRT64 or CLANG64 are technically modern and usable, but the project's `CMakeLists.txt` is configured to look for libraries under `C:\msys64\mingw64`. This guide therefore assumes the MinGW64 environment. Do not use the generic MSYS2 shell or Windows PowerShell for the commands below.

## Step 2 — Update MSYS2

Update the package database and installed packages. If MSYS2 asks you to close the terminal after a core-system upgrade, close it, reopen **MSYS2 MinGW 64-bit** and run the command again.

```
pacman -Syu
# If prompted to close the terminal, reopen MSYS2 MinGW64 and run again:
pacman -Syu
```

## Step 3 — Install toolchain and base libraries

Install the compiler, CMake, Ninja, Boost, Eigen, tinyxml2, libzip, gnuplot, GDB and Python. The `--needed` flag skips packages that are already installed.

```
pacman -S --needed \
  git \
  mingw-w64-x86_64-gcc \
  mingw-w64-x86_64-cmake \
  mingw-w64-x86_64-ninja \
  mingw-w64-x86_64-boost \
  mingw-w64-x86_64-eigen3 \
  mingw-w64-x86_64-tinyxml2 \
  mingw-w64-x86_64-libzip \
  mingw-w64-x86_64-gnuplot \
  mingw-w64-x86_64-gdb \
  mingw-w64-x86_64-python
```

### Toolchain verification

```
git --version
g++ --version
cmake --version
ninja --version
gnuplot --version
gdb --version
python --version
```

| Check           | Expected output                           |
|-----------------|-------------------------------------------|
| git --version   | Prints an installed Git version.          |
| g++ --version   | Reports GCC from the MinGW64 toolchain.   |
| cmake --version | Prints the MSYS2-installed CMake version. |

|                  |                            |
|------------------|----------------------------|
| ninja --version  | Prints the Ninja version.  |
| python --version | Prints a Python 3 version. |

## Step 4 — Install VTK and useful dependencies

To avoid CMake errors caused by missing imported VTK targets, install VTK together with the optional dependencies most commonly required by its modules.

### ■ Disk space notice

This step downloads several gigabytes of packages (typically 3–5 GB of disk usage after installation). They are optional VTK dependencies and many of them may never be exercised by AERO++ directly. Installing them in bulk is the simplest way to avoid CMake errors about missing VTK components. If disk space is limited, skip this step and install individual packages only when CMake reports them as missing.

```
pacman -S --needed \  
  mingw-w64-x86_64-vtk \  
  mingw-w64-x86_64-nlohmann-json \  
  mingw-w64-x86_64-fast_float \  
  mingw-w64-x86_64-utf8cpp \  
  mingw-w64-x86_64-cli11 \  
  mingw-w64-x86_64-openvr \  
  mingw-w64-x86_64-hdf5 \  
  mingw-w64-x86_64-netcdf \  
  mingw-w64-x86_64-libogg \  
  mingw-w64-x86_64-libtheora \  
  mingw-w64-x86_64-cgns \  
  mingw-w64-x86_64-gl2ps \  
  mingw-w64-x86_64-proj \  
  mingw-w64-x86_64-qt6-declarative \  
  mingw-w64-x86_64-openslide \  
  mingw-w64-x86_64-adios2
```

Verify the installed VTK-related packages:

```
pacman -Q | grep -Ei "vtk|nlohmann|fast_float|utf8|cli11|openvr|hdf5|netcdf|cgns|gl2ps|proj|qt6-declara
```

## Step 5 — Clone AERO++ into the MSYS2 home

### ■ Critical: do not clone under C:\Users\...\Documents

On Windows 10/11 the Documents folder is synchronised with OneDrive by default. OneDrive can silently delete or convert into cloud placeholders the files produced by CMake and Ninja during configuration, causing CMakeCache.txt to disappear after an apparently successful configure. The MSYS2 home (~, that is C:\msys64\home\<user>) is not touched by OneDrive and is the recommended location for student builds.

Clone the repository into your MSYS2 home. **The folder name is up to you:** by default `git clone` uses the repository name (for example `AEROPlusPlus`), but you can pass an explicit destination as the last argument. The rest of this guide will refer to the project root as `~/AEROPlusPlus`; replace it everywhere with the name you actually chose.

### Default clone (folder named after the repo)

```
cd ~
git clone https://github.com/Am3Software/AEROPlusPlus.git
cd ~/AEROPlusPlus
pwd
ls
```

### Custom folder name (replace myproject with any name you prefer):

```
cd ~
git clone https://github.com/Am3Software/AEROPlusPlus.git myproject
cd ~/myproject
pwd
ls
```

| Check            | Expected output                                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pwd</code> | Must show <code>/home/&lt;user&gt;/&lt;your-folder&gt;</code> .                                                                           |
| <code>ls</code>  | Must list <code>CMakeLists.txt</code> , <code>include/</code> , <code>test/</code> , <code>README.md</code> and the data folders, if any. |

### If runtime data folders are missing

If `AircraftSettings`, `FuselagePreset` or `NacellePreset` are not present locally, fetch them from `origin/main` without performing a full pull.

```
git fetch origin
git checkout origin/main -- AircraftSettings FuselagePreset NacellePreset
ls
```

## Step 6 — Vendor OpenXLSX at the correct commit

AERO++ relies on OpenXLSX APIs that are only available at a specific commit. The tested, known-good revision is 5723411d47643ce3b5b9994064c26ca8cd841f13, described as v0.3.2-227-g5723411.

```
cd ~/AEROPlusPlus
mkdir -p third_party
git clone --no-checkout https://github.com/troldal/OpenXLSX.git third_party/OpenXLSX
git -C third_party/OpenXLSX checkout 5723411d47643ce3b5b9994064c26ca8cd841f13
```

### ■ Note: detached HEAD

After the checkout, Git will report *HEAD detached at 5723411*. This is normal and expected for this procedure: you have pinned the dependency to an exact commit rather than a moving branch.

## OpenXLSX verification

```
git -C third_party/OpenXLSX describe --tags --always --dirty
git -C third_party/OpenXLSX rev-parse HEAD
git -C third_party/OpenXLSX status --short
```

| Check          | Expected output                                           |
|----------------|-----------------------------------------------------------|
| describe       | Must print v0.3.2-227-g5723411 without the -dirty suffix. |
| rev-parse      | Must print 5723411d47643ce3b5b9994064c26ca8cd841f13.      |
| status --short | Must print nothing (clean working tree).                  |

## Verify the API surface used by AERO++

```
grep -R "XLForceOverwrite" -n third_party/OpenXLSX/OpenXLSX 2>/dev/null | head -5
grep -R "column.*std::string\|column.*string" -n third_party/OpenXLSX/OpenXLSX 2>/dev/null | head -10
```

| Check               | Expected output                |
|---------------------|--------------------------------|
| XLForceOverwrite    | Must appear in XLDocument.hpp. |
| column(std::string) | Must appear in XLSheet.hpp.    |

## If OpenXLSX is reported as 'dirty' or you get a Permission denied error

```
chmod -R u+w third_party/OpenXLSX
rm -rf third_party/OpenXLSX
git clone --no-checkout https://github.com/troldal/OpenXLSX.git third_party/OpenXLSX
git -C third_party/OpenXLSX checkout 5723411d47643ce3b5b9994064c26ca8cd841f13
```

## Step 7 — Sanity-check the CMakeLists.txt

### ■ Optional step

These checks are not part of the build. Run them only if you suspect your local CMakeLists.txt is out of date. If any check produces no output, fix it by updating your local repository (`git pull`) rather than editing the file by hand.

The student-facing CMakeLists.txt should have three essential traits: a C/C++ project declaration, a restricted list of VTK components, and a vendored OpenXLSX subdirectory.

### Quick checks

```
grep -n "project(AEROPlusPlus" CMakeLists.txt
grep -n "AEROPP_VTK_COMPONENTS\\|find_package(VTK" CMakeLists.txt
grep -n "OPENXLSX_LOCAL_DIR\\|add_subdirectory" CMakeLists.txt
grep -n "AEROPlusPlusOpenXLSXCompat" CMakeLists.txt
```

### Conceptual expected output

```
project(AEROPlusPlus LANGUAGES C CXX)
find_package(VTK CONFIG REQUIRED COMPONENTS ${AEROPP_VTK_COMPONENTS})
set(OPENXLSX_LOCAL_DIR "${CMAKE_CURRENT_SOURCE_DIR}/third_party/OpenXLSX")
add_subdirectory("${OPENXLSX_LOCAL_DIR}" ...)
AEROPlusPlusOpenXLSXCompat.h
```

The OpenXLSX compatibility header AEROPlusPlusOpenXLSXCompat.h must include `<cstdint>` and `<algorithm>`. The robust CMakeLists.txt generates it automatically inside the build directory and injects it via `-include`; you should not edit or move it.

## Step 8 — Configure CMake

From this point on, always work from your project root (~/`AEROPlusPlus` or whatever you named it). To avoid inconsistent build directories, remove `build` before re-running the configure.

```
cd ~/AEROPlusPlus
rm -rf build
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_POLICY_VERSION_MINIMUM=3.5 \
  -DCMAKE_EXPORT_COMPILE_COMMANDS=ON \
  -Wno-dev
```

### Expected final lines

```
-- Configuring done
-- Generating done
-- Build files have been written to: .../<your-folder>/build
```

### Verify the build directory

```
ls -l build/CMakeCache.txt
ls -l build/build.ninja
ls -l build/compile_commands.json
```

#### ■ If `CMakeCache.txt` is missing

The configure step failed or was interrupted. **Do not run the build** in that state, and **never create `CMakeCache.txt` manually**. Remove the `build` directory and re-run the configure while reading the first CMake Error reported in the output: all subsequent errors are usually cascading from it.

## Step 9 — Build (full or single target)

### Full build

The `-j` flag lets Ninja parallelise across all available cores, which dramatically shortens the first build (which must compile the bundled OpenXLSX). You can replace `-j` with `-j4` (or any other number) to limit core usage while you keep using the machine.

```
cd ~/AEROPlusPlus
cmake --build build -j
```

The build is successful when it finishes without any FAILED lines and without fatal error messages. Warnings such as UNICODEREDefined are not blocking.

### Single-target build

To iterate faster on a single executable (for example SpitFire\_Test):

```
cmake --build build --target SpitFire_Test -j
```

### Confirm the executable was produced

```
ls -l build/SpitFire_Test.exe
```

## Step 10 — Run an executable

Run the binary from the project root. Many tests generate files inside the build directory and rely on the runtime data folders that CMake copied there during configuration (AircraftSettings, ExcelFiles, FuselagePreset, NacellePreset).

```
cd ~/AEROPlusPlus
./build/SpitFire_Test.exe
```

#### ■ OpenVSP dependency

Some tests invoke `vspscript.exe` or `vspaero.exe`. They must be reachable through the system PATH. AERO++ can compile successfully even if OpenVSP is missing; the error will only surface at runtime when an executable tries to call it.

## Quick reference — full copy-paste sequence

This sequence summarises the procedure on a freshly updated MSYS2 installation. Replace the project folder name as needed.

```
# 1) Update MSYS2
pacman -Syu

# 2) Base toolchain and libraries
pacman -S --needed \
  git mingw-w64-x86_64-gcc mingw-w64-x86_64-cmake mingw-w64-x86_64-ninja \
  mingw-w64-x86_64-boost mingw-w64-x86_64-eigen3 mingw-w64-x86_64-tinyxml2 \
  mingw-w64-x86_64-libzip mingw-w64-x86_64-gnuplot mingw-w64-x86_64-gdb \
  mingw-w64-x86_64-python

# 3) VTK and useful optional dependencies
pacman -S --needed \
  mingw-w64-x86_64-vtk mingw-w64-x86_64-nlohmann-json mingw-w64-x86_64-fast_float \
  mingw-w64-x86_64-utf8cpp mingw-w64-x86_64-cli11 mingw-w64-x86_64-openvr \
  mingw-w64-x86_64-hdf5 mingw-w64-x86_64-netcdf mingw-w64-x86_64-libogg \
  mingw-w64-x86_64-libtheora mingw-w64-x86_64-cgns mingw-w64-x86_64-gl2ps \
  mingw-w64-x86_64-proj mingw-w64-x86_64-qt6-declarative \
  mingw-w64-x86_64-openslide mingw-w64-x86_64-adios2

# 4) Clone AERO++ into your MSYS2 home (folder name is up to you)
cd ~
git clone https://github.com/Am3Software/AEROPlusPlus.git
cd ~/AEROPlusPlus

# 5) Vendor OpenXLSX at the pinned commit
mkdir -p third_party
git clone --no-checkout https://github.com/troldal/OpenXLSX.git third_party/OpenXLSX
git -C third_party/OpenXLSX checkout 5723411d47643ce3b5b9994064c26ca8cd841f13

# 6) Configure and build
rm -rf build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_POLICY_VERSION_MINIMUM=3.5 -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -Wno-dev
cmake --build build -j
```

# Troubleshooting

## 1. build/CMakeCache.txt is missing

The configure step did not complete successfully. Never create CMakeCache.txt by hand — it is generated by CMake only when configuration succeeds.

```
cd ~/AEROPlusPlus
rm -rf build
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_POLICY_VERSION_MINIMUM=3.5 \
  -DCMAKE_EXPORT_COMPILE_COMMANDS=ON \
  -Wno-dev 2>&1 | tee configure.log
grep -n "CMake Error" configure.log
tail -80 configure.log
```

## 2. CMakeCache.txt keeps disappearing after a successful configure

The build directory is inside a OneDrive-synchronised path (typically C:\Users\<user>\Documents\ ) or inside a Windows Defender Controlled Folder. Move the project to your MSYS2 home and re-run the configure:

```
mv /c/Users/<user>/Documents/<project> ~/
cd ~/<project>
rm -rf build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_POLICY_VERSION_MINIMUM=3.5 -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -Wno-dev
```

## 3. build.ninja exists but CMakeCache.txt does not

The build directory is incomplete; the configure was interrupted or failed late. Recreate the directory:

```
rm -rf build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_POLICY_VERSION_MINIMUM=3.5 -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -Wno-dev
```

## 4. OpenXLSX error: XLForceOverwrite not declared

You are using a wrong or too-old version of OpenXLSX. Reinstall the vendored copy at the pinned commit:

```
rm -rf third_party/OpenXLSX
git clone --no-checkout https://github.com/troldal/OpenXLSX.git third_party/OpenXLSX
git -C third_party/OpenXLSX checkout 5723411d47643ce3b5b9994064c26ca8cd841f13
```

## 5. OpenXLSX reported as dirty

```
git -C third_party/OpenXLSX describe --tags --always --dirty
# If -dirty appears:
chmod -R u+w third_party/OpenXLSX
rm -rf third_party/OpenXLSX
# Then re-clone with --no-checkout as in Step 6
```

## 6. VTK errors about missing dependencies

Make sure your CMakeLists.txt uses the AERO\_PP\_VTK\_COMPONENTS list (a restricted set of components) and install the packages from Step 4. Loading all of VTK with find\_package(VTK CONFIG REQUIRED) without explicit components is much more likely to trigger errors about optional dependencies.

## 7. Permission denied on Excel files or Windows folders

The most frequent causes on Windows are: OneDrive holding a lock on the file, Excel / VS Code / Explorer keeping it open, or an antivirus Controlled Folder policy. Close any application that may be holding the file, move the project to your MSYS2 home, and rebuild.

## 8. Runtime folders missing after clone

```
git fetch origin
git checkout origin/main -- AircraftSettings FuselagePreset NacellePreset
```

## 9. UNICODE redefined warning

This is only a warning, not a blocking error. It is emitted when the same macro is defined both on the command line and inside the sources. You can safely ignore it.

## Final rules

- Always use the **MSYS2 MinGW 64-bit** shell.
- Always clone the project inside `~` (your MSYS2 home). The folder name is your choice.
- Never place the project inside `C:\Users\<user>\Documents\` while OneDrive is active.
- Never create `CMakeCache.txt` manually. It is produced by CMake when configuration succeeds.
- Never run a build if `CMakeCache.txt` does not exist in the `build/` directory.
- Keep OpenXLSX pinned to `5723411d47643ce3b5b9994064c26ca8cd841f13` and the working tree clean (no `-dirty` suffix).
- If configuration fails, read the **first** CMake Error in the output and act on that one. Do not edit project files at random.